

GNOT: General Neural Operator Transformer

Mathematical Formulation and PyTorch Reference Implementation

1 Introduction

The General Neural Operator Transformer (GNOT) is a transformer-based neural operator designed to learn mappings between function spaces.

The central objective is

$$\mathcal{G} : a(\cdot) \mapsto u(\cdot), \quad (1)$$

where a is an input function and u is the corresponding output function, such as the solution of a parametrized PDE.

GNOT combines operator-learning ideas with transformer attention. Its architecture is designed to capture long-range interactions between function values while retaining the flexibility of neural operators.

2 Discretized Operator Learning

Let the input function be sampled at

$$X = \{x_1, \dots, x_N\}. \quad (2)$$

The input field is

$$a_i = a(x_i). \quad (3)$$

The initial representation is

$$z_i^0 = P_{\text{in}}[a_i; x_i]. \quad (4)$$

Thus,

$$Z^0 = [z_1^0, \dots, z_N^0]^T \in \mathbb{R}^{N \times d}. \quad (5)$$

The transformer maps

$$Z^0 \rightarrow Z^1 \rightarrow \dots \rightarrow Z^L. \quad (6)$$

The final prediction is

$$\hat{u}_i = P_{\text{out}}(z_i^L). \quad (7)$$

3 Attention as an Integral Operator

A continuous integral operator has the form

$$(\mathcal{K}v)(x) = \int_D \kappa(x, y) v(y) dy. \quad (8)$$

A transformer attention mechanism can be interpreted as a learned discretization of such a nonlocal interaction.

For query locations x_i and source locations x_j ,

$$(\mathcal{K}v)_i \approx \sum_j A_{ij} v_j. \quad (9)$$

The attention weights are learned from query and key representations.

4 Multi-Head Attention

For feature matrix

$$Z \in \mathbb{R}^{N \times d}, \quad (10)$$

define

$$Q = ZW_Q, \quad (11)$$

$$K = ZW_K, \quad (12)$$

$$V = ZW_V. \quad (13)$$

The scaled dot-product attention is

$$A = \text{softmax} \left(\frac{QK^T}{\sqrt{d_h}} \right). \quad (14)$$

The output is

$$Y = AV. \quad (15)$$

For H attention heads,

$$\text{MHA}(Z) = \text{Concat}(Y_1, \dots, Y_H) W_O. \quad (16)$$

5 Cross-Function Attention

A neural operator may involve two different sets of points.

Let

$$X_q = \{x_1^q, \dots, x_{N_q}^q\} \quad (17)$$

be query points and

$$X_k = \{x_1^k, \dots, x_{N_k}^k\} \quad (18)$$

be source points.

The query representation is

$$Q = Z_q W_Q, \quad (19)$$

while the key and value representations are

$$K = Z_k W_K, \quad (20)$$

$$V = Z_v W_V. \quad (21)$$

The cross-attention operator is

$$Y = \text{softmax} \left(\frac{QK^T}{\sqrt{d_h}} \right) V. \quad (22)$$

This gives an explicit mechanism for mapping information from one function representation to another.

6 Geometric Encoding

Because neural operators act on functions defined over domains, spatial coordinates are incorporated into the representation.

Let

$$x_i \in \mathbb{R}^{d_x}. \quad (23)$$

A coordinate embedding is

$$e_i = P_x(x_i). \quad (24)$$

The complete feature becomes

$$z_i = P_a(a_i) + P_x(x_i). \quad (25)$$

A more general concatenated representation is

$$z_i = P_{\text{in}}[a_i; x_i]. \quad (26)$$

7 Transformer Operator Layer

A GNOT transformer layer can be represented abstractly as

$$Z^{l+1} = Z^l + \text{Attention} \left(\text{LN}(Z^l) \right). \quad (27)$$

The feed-forward component is then

$$Z^{l+1} = Z^{l+1} + \text{MLP} \left(\text{LN}(Z^{l+1}) \right). \quad (28)$$

Thus,

$$Z^{l+1} = Z^l + \text{Attn}(\text{LN}(Z^l)) + \text{MLP} \left(\text{LN} \left[Z^l + \text{Attn}(\text{LN}(Z^l)) \right] \right). \quad (29)$$

8 Operator Interpretation

The attention matrix

$$A_{ij} = \frac{\exp(q_i^T k_j / \sqrt{d_h})}{\sum_m \exp(q_i^T k_m / \sqrt{d_h})} \quad (30)$$

acts as a learned discrete kernel.

Consequently,

$$y_i = \sum_j A_{ij} v_j \quad (31)$$

is analogous to

$$(\mathcal{K}v)(x_i) = \int \kappa(x_i, y) v(y) dy. \quad (32)$$

This provides the connection between transformer attention and neural operators.

9 Latent Representation

Instead of repeatedly operating directly on the full input resolution, operator-transformer architectures can use a latent representation.

Let

$$Z \in \mathbb{R}^{N \times d}. \quad (33)$$

A latent projection gives

$$H = P_{\text{latent}}(Z), \quad (34)$$

where

$$H \in \mathbb{R}^{M \times d}, \quad M < N. \quad (35)$$

Attention can then operate between input and latent representations.

This reduces the cost of global interactions.

10 Query-Key-Value Operator Form

Suppose

$$Q = \phi_q(x), \quad (36)$$

$$K = \phi_k(y), \quad (37)$$

and

$$V = \phi_v(v(y)). \quad (38)$$

Then

$$(\mathcal{K}_\theta v)(x) \approx \int \text{softmax}_y \left(\frac{\phi_q(x)^T \phi_k(y)}{\sqrt{d_h}} \right) \phi_v(v(y)) dy. \quad (39)$$

The corresponding discrete expression is

$$y_i = \sum_j \frac{\exp(q_i^T k_j / \sqrt{d_h})}{\sum_m \exp(q_i^T k_m / \sqrt{d_h})} v_j. \quad (40)$$

11 Output Projection

After L transformer layers,

$$Z^L = \text{GNOT}_\theta(Z^0). \quad (41)$$

The output field is obtained through

$$\hat{u}_i = P_{\text{out}}(z_i^L). \quad (42)$$

Therefore,

$$\mathcal{G}_\theta(a)(x_i) = P_{\text{out}}[\text{GNOT}_\theta(P_{\text{in}}[a(x); x])_i]. \quad (43)$$

12 Training Objective

Given

$$\{a^{(n)}, u^{(n)}\}_{n=1}^N, \quad (44)$$

the supervised loss is

$$\mathcal{L} = \frac{1}{N} \sum_{n=1}^N \left\| \mathcal{G}_\theta(a^{(n)}) - u^{(n)} \right\|_2^2. \quad (45)$$

The relative error is

$$\mathcal{L}_{\text{rel}} = \frac{1}{N} \sum_{n=1}^N \frac{\|\hat{u}^{(n)} - u^{(n)}\|_2}{\|u^{(n)}\|_2 + \epsilon}. \quad (46)$$

13 Computational Complexity

For N tokens and feature dimension d , standard self-attention has complexity

$$\mathcal{O}(N^2 d). \quad (47)$$

If attention is performed between N input points and M latent points, the dominant interaction becomes

$$\mathcal{O}(NMd), \quad (48)$$

where

$$M \ll N. \quad (49)$$

Thus latent or factorized formulations can substantially reduce the cost of operator learning at high spatial resolution.

14 PyTorch Reference Implementation

```
1
2 import torch
3 import torch.nn as nn
4
5
6 class CrossAttention(nn.Module):
7
8     def __init__(
9         self,
10         dim,
11         heads=4
12     ):
13         super().__init__()
14
15         assert dim % heads == 0
16
17         self.dim = dim
18         self.heads = heads
19         self.head_dim = dim // heads
20
21         self.q_proj = nn.Linear(
22             dim,
23             dim
24         )
25
26         self.k_proj = nn.Linear(
27             dim,
28             dim
29         )
30
31         self.v_proj = nn.Linear(
32             dim,
33             dim
34         )
35
36         self.out_proj = nn.Linear(
37             dim,
38             dim
39         )
40
41     def forward(
42         self,
43         query,
44         context
45     ):
46
47         B, Nq, C = query.shape
48         Nk = context.shape[1]
49
50         q = self.q_proj(query)
51         k = self.k_proj(context)
52         v = self.v_proj(context)
53
54         q = q.view(
55             B,
56             Nq,
```

```

57         self.heads,
58         self.head_dim
59     ).transpose(1, 2)
60
61     k = k.view(
62         B,
63         Nk,
64         self.heads,
65         self.head_dim
66     ).transpose(1, 2)
67
68     v = v.view(
69         B,
70         Nk,
71         self.heads,
72         self.head_dim
73     ).transpose(1, 2)
74
75     scores = torch.matmul(
76         q,
77         k.transpose(-2, -1)
78     )
79
80     scores = (
81         scores
82         / self.head_dim ** 0.5
83     )
84
85     attention = torch.softmax(
86         scores,
87         dim=-1
88     )
89
90     out = torch.matmul(
91         attention,
92         v
93     )
94
95     out = out.transpose(
96         1,
97         2
98     ).contiguous()
99
100    out = out.view(
101        B,
102        Nq,
103        C
104    )
105
106    return self.out_proj(out)
107
108
109 class GNOTBlock(nn.Module):
110
111     def __init__(
112         self,
113         dim,
114         heads=4,

```

```

115         mlp_ratio=4
116     ):
117         super().__init__()
118
119         self.norm1 = nn.LayerNorm(dim)
120
121         self.attention = CrossAttention(
122             dim,
123             heads
124         )
125
126         self.norm2 = nn.LayerNorm(dim)
127
128         hidden = (
129             dim * mlp_ratio
130         )
131
132         self.mlp = nn.Sequential(
133             nn.Linear(
134                 dim,
135                 hidden
136             ),
137             nn.GELU(),
138             nn.Linear(
139                 hidden,
140                 dim
141             )
142         )
143
144     def forward(
145         self,
146         x,
147         context
148     ):
149
150         x_norm = self.norm1(x)
151
152         x = (
153             x
154             +
155             self.attention(
156                 x_norm,
157                 context
158             )
159         )
160
161         x = (
162             x
163             +
164             self.mlp(
165                 self.norm2(x)
166             )
167         )
168
169         return x
170
171
172 class GNOT(nn.Module):

```



```

173
174     def __init__(
175         self,
176         in_channels,
177         out_channels,
178         coord_dim=2,
179         dim=64,
180         depth=4,
181         heads=4,
182         latent_points=128
183     ):
184         super().__init__()
185
186         self.latent_points = latent_points
187
188         self.input_projection = nn.Linear(
189             in_channels + coord_dim,
190             dim
191         )
192
193         self.latent_tokens = nn.Parameter(
194             torch.randn(
195                 1,
196                 latent_points,
197                 dim
198             )
199         )
200
201         self.encoder = GNOTBlock(
202             dim,
203             heads
204         )
205
206         self.blocks = nn.ModuleList([
207             GNOTBlock(
208                 dim,
209                 heads
210             )
211             for _ in range(depth)
212         ])
213
214         self.output_projection = nn.Sequential(
215             nn.Linear(
216                 dim,
217                 dim
218             ),
219             nn.GELU(),
220             nn.Linear(
221                 dim,
222                 out_channels
223             )
224         )
225
226     def forward(
227         self,
228         coords,
229         input_field
230     ):

```

```

231
232     B = coords.shape[0]
233
234     x = torch.cat(
235         [
236             coords,
237             input_field
238         ],
239         dim=-1
240     )
241
242     x = self.input_projection(x)
243
244     latent = self.latent_tokens.expand(
245         B,
246         -1,
247         -1
248     )
249
250     # Encode input points into latent representation
251     latent = self.encoder(
252         latent,
253         x
254     )
255
256     # Latent transformer processing
257     for block in self.blocks:
258
259         latent = block(
260             latent,
261             latent
262         )
263
264     # Decode latent representation
265     decoded = self.encoder(
266         x,
267         latent
268     )
269
270     return self.output_projection(
271         decoded
272     )
273
274
275 if __name__ == "__main__":
276
277     device = (
278         "cuda"
279         if torch.cuda.is_available()
280         else "cpu"
281     )
282
283     batch_size = 4
284     num_points = 1024
285
286     coords = torch.rand(
287         batch_size,
288         num_points,

```

```

289         2,
290         device=device
291     )
292
293     input_field = torch.randn(
294         batch_size,
295         num_points,
296         1,
297         device=device
298     )
299
300     model = GNOT(
301         in_channels=1,
302         out_channels=1,
303         coord_dim=2,
304         dim=64,
305         depth=4,
306         heads=4,
307         latent_points=128
308     ).to(device)
309
310     output = model(
311         coords,
312         input_field
313     )
314
315     print(
316         "Input:",
317         input_field.shape
318     )
319
320     print(
321         "Output:",
322         output.shape
323     )

```

15 Training

```

1
2 model = GNOT(
3     in_channels=1,
4     out_channels=1,
5     coord_dim=2,
6     dim=64,
7     depth=4,
8     heads=4,
9     latent_points=128
10 ).to(device)
11
12 optimizer = torch.optim.Adam(
13     model.parameters(),
14     lr=1e-3
15 )
16
17 for epoch in range(100):
18

```

```

19     model.train()
20
21     optimizer.zero_grad()
22
23     prediction = model(
24         coords,
25         input_field
26     )
27
28     loss = torch.mean(
29         (prediction - target) ** 2
30     )
31
32     loss.backward()
33
34     optimizer.step()
35
36     if epoch % 10 == 0:
37
38         print(
39             f"Epoch {epoch} | "
40             f"Loss = {loss.item():.6e}"
41         )

```

16 GNOT versus FNO and GNO

The three architectures parameterize nonlocal interactions differently.

FNO:

$$\mathcal{K}v = \mathcal{F}^{-1} [R(k)\mathcal{F}(v)]. \quad (50)$$

GNO:

$$\mathcal{K}v(x_i) = \sum_j \kappa_\phi(x_i, x_j, v_i, v_j) v_j w_j. \quad (51)$$

GNOT:

$$\mathcal{K}v(x_i) \approx \sum_j A_{ij} V_j, \quad (52)$$

where

$$A_{ij} = \text{softmax}_j \left(\frac{q_i^T k_j}{\sqrt{d_h}} \right). \quad (53)$$

Thus attention provides a learned, data-dependent kernel.

Property	FNO	GNO	GNOT
Core operation	FFT	Kernel integral	Attention
Geometry	Grid	Point cloud	Point/function tokens
Kernel	Fourier weights	MLP kernel	Q-K attention
Global interaction	Spectral	Graph-dependent	Attention
Irregular domains	Limited	Strong	Strong

17 Summary

GNOT combines the operator-learning perspective with transformer attention.

Its essential operation can be expressed as

$$\mathcal{K}_\theta(v)(x_i) = \sum_j \text{softmax}_j \left(\frac{q_\theta(x_i)^T k_\theta(x_j)}{\sqrt{d_h}} \right) v_\theta(x_j). \quad (54)$$

The complete architecture is

$$a(x) \rightarrow \text{Input Embedding} \rightarrow \text{Operator Attention} \rightarrow \text{Transformer Blocks} \rightarrow \text{Output Projection} \rightarrow u(x). \quad (55)$$

The important conceptual point is that attention is not merely being used as a conventional sequence model. It serves as a learned nonlocal kernel for approximating an operator between function spaces.

References

[1] J. Li, et al.,

GNOT: A General Neural Operator Transformer for Operator Learning,
2023.